

PS2

Alejandro De La Torre

September 2025

Due: 11:59pm Central, Friday 09/26/2025. Submit a PDF to Canvas.

1 Population Regression vs. CEF

In lecture 2, we considered three objects: y_i , $E[y_i | x_i]$, and $x_i'\beta^*$, the “infeasible” (population) OLS target. We can write

$$y_i = x_i'\beta^* + u_i, \quad \beta^* = \arg \min_b \mathbb{E}[(y_i - x_i'b)^2].$$

1. Show that $\mathbb{E}[x_i u_i] = 0$.
2. Show that $\beta^* = \left(\mathbb{E}[x_i x_i']\right)^{-1} \mathbb{E}[x_i y_i]$.
3. Show that $\mathbb{E}[u_i | x_i] = 0 \Rightarrow \mathbb{E}[x_i u_i] = 0$. (Hint: Law of Iterated Expectations.)
4. List two cases where $\mathbb{E}[u_i | x_i] = 0$. Clearly state your assumption and explain why it implies $\mathbb{E}[u_i | x_i] = 0$.

Solution.

- (1) We want to show that $\mathbb{E}[x_i u_i] = 0$.

Recall that

$$u_i = y_i - x_i'\beta^*,$$

where β^* solves

$$\beta^* = \arg \min_b \mathbb{E}[(y_i - x_i'b)^2].$$

Define the objective

$$Q(b) = \mathbb{E}[(y_i - x_i'b)^2].$$

Taking the gradient with respect to b gives

$$\frac{\partial Q(b)}{\partial b} = \mathbb{E}[-2x_i(y_i - x_i'b)].$$

At $b = \beta^*$, the first-order condition requires

$$\mathbb{E}[x_i(y_i - x_i'\beta^*)] = 0.$$

Distributing the terms inside the expectation:

$$\mathbb{E}[x_i y_i] - \mathbb{E}[x_i x_i']\beta^* = 0.$$

Equivalently,

$$\mathbb{E}[x_i u_i] = 0.$$

Thus, by construction of the population regression, the regressors are orthogonal to the residuals. $\square$

(2) We want to show that

$$\beta^* = \mathbb{E}[x_i x_i']^{-1} \mathbb{E}[x_i y_i].$$

Recall that β^* is defined as the minimizer of the population least squares problem:

$$\beta^* = \arg \min_b \mathbb{E}[(y_i - x_i' b)^2].$$

Define the objective function

$$Q(b) = \mathbb{E}[(y_i - x_i' b)^2].$$

Taking the partial derivative with respect to b gives

$$\frac{\partial Q(b)}{\partial b} = \mathbb{E}[-2x_i(y_i - x_i' b)].$$

Setting the first-order condition to zero at $b = \beta^*$:

$$\mathbb{E}[x_i(y_i - x_i'\beta^*)] = 0.$$

Distributing the terms inside the expectation:

$$\mathbb{E}[x_i y_i] - \mathbb{E}[x_i x_i']\beta^* = 0.$$

Rearranging:

$$\mathbb{E}[x_i x_i']\beta^* = \mathbb{E}[x_i y_i].$$

Assuming $\mathbb{E}[x_i x_i']$ is invertible, we obtain

$$\beta^* = \mathbb{E}[x_i x_i']^{-1} \mathbb{E}[x_i y_i].$$

$\square$

- (3) Assume $\mathbb{E}[u_i | x_i] = 0$. We want to show $\mathbb{E}[x_i u_i] = 0$.

Apply the Law of Iterated Expectations (LIE):

$$\mathbb{E}[x_i u_i] = \mathbb{E}[\mathbb{E}[x_i u_i | x_i]].$$

Because x_i is measurable with respect to $\sigma(x_i)$, it is constant inside the inner conditional expectation; hence

$$\mathbb{E}[x_i u_i | x_i] = x_i \mathbb{E}[u_i | x_i].$$

Substitute the assumption $\mathbb{E}[u_i | x_i] = 0$:

$$\mathbb{E}[x_i u_i | x_i] = x_i \cdot 0 = 0.$$

Take unconditional expectations:

$$\mathbb{E}[x_i u_i] = \mathbb{E}[0] = 0.$$

□

- (4) **Two situations implying $\mathbb{E}[u_i | x_i] = 0$.**

- i. *Linear CEF (correctly specified linear model).* Assume the conditional expectation function is linear:

$$m(x) \equiv \mathbb{E}[y_i | x_i = x] = x' \beta_0 \quad \text{for some } \beta_0.$$

Define the error as the deviation from the CEF:

$$u_i = y_i - m(x_i) = y_i - x_i' \beta_0.$$

Then

$$\mathbb{E}[u_i | x_i] = \mathbb{E}[y_i - m(x_i) | x_i] = \mathbb{E}[y_i | x_i] - m(x_i) = m(x_i) - m(x_i) = 0.$$

Hence $\mathbb{E}[u_i | x_i] = 0$ when the CEF is linear (the “correctly specified” linear model).

- ii. *Mean-zero error independent of x_i (e.g., randomized x_i).* Suppose the data are generated by

$$y_i = x_i' \beta_0 + v_i,$$

where v_i has mean zero and is independent of x_i :

$$\mathbb{E}[v_i] = 0, \quad v_i \perp\!\!\!\perp x_i.$$

Take $u_i \equiv v_i$. Independence implies conditional expectations equal unconditional ones, so

$$\mathbb{E}[u_i | x_i] = \mathbb{E}[v_i | x_i] = \mathbb{E}[v_i] = 0.$$

A canonical example is a randomized experiment where treatment (an element of x_i) is assigned independently of potential outcomes.

2 Regressions with Dummies

Suppose we have a sample of size N formed by two groups, 0 and 1. Let D_i be a dummy with $D_i = 1$ indicating group 1. Let N_0 and N_1 be the group sizes ($N = N_0 + N_1$).

- a) Consider the OLS model $y_i = \alpha + \beta D_i + u_i$. Using the normal equations for $(\hat{\alpha}, \hat{\beta})$, show that

$$\hat{\alpha} = \frac{1}{N_0} \sum_{i \in 0} y_i, \quad \hat{\alpha} + \hat{\beta} = \frac{1}{N_1} \sum_{i \in 1} y_i.$$

- b) Now suppose there are 3 groups 0, 1, 2 with dummies $D_{1i} = 1\{i \in 1\}$ and $D_{2i} = 1\{i \in 2\}$ and model

$$y_i = \alpha + \beta_1 D_{1i} + \beta_2 D_{2i} + u_i.$$

- i. Write the first-order conditions (FOC) that define the OLS estimator.
- ii. Use the FOC to show

$$\hat{\alpha} = \frac{1}{N_0} \sum_{i \in 0} y_i, \quad \hat{\alpha} + \hat{\beta}_1 = \frac{1}{N_1} \sum_{i \in 1} y_i, \quad \hat{\alpha} + \hat{\beta}_2 = \frac{1}{N_2} \sum_{i \in 2} y_i.$$

Solution.

- (a) **Using the OLS first-order conditions.** Model: $y_i = \alpha + \beta D_i + u_i$ with $D_i \in \{0, 1\}$, $D_i = 1$ for group 1 and 0 otherwise. Let $N_0 = \#\{i : D_i = 0\}$ and $N_1 = \#\{i : D_i = 1\}$ (so $N = N_0 + N_1$).

Minimize the sum of squared residuals

$$S(\alpha, \beta) = \sum_{i=1}^N (y_i - \alpha - \beta D_i)^2.$$

FOC w.r.t. α :

$$\frac{\partial S}{\partial \alpha} = -2 \sum_{i=1}^N (y_i - \alpha - \beta D_i) = 0 \quad \implies \quad \sum_{i=1}^N y_i = N\alpha + \beta \sum_{i=1}^N D_i = N\alpha + \beta N_1. \quad (\text{NE-}\alpha)$$

FOC w.r.t. β :

$$\begin{aligned} \frac{\partial S}{\partial \beta} &= -2 \sum_{i=1}^N D_i (y_i - \alpha - \beta D_i) = 0 \\ \implies \sum_{i=1}^N D_i y_i &= \alpha \sum_{i=1}^N D_i + \beta \sum_{i=1}^N D_i^2 = \alpha N_1 + \beta N_1, \end{aligned} \quad (\text{NE-}\beta)$$

using $D_i^2 = D_i$. Since $D_i y_i = y_i$ only when $i \in 1$,

$$\sum_{i \in 1} y_i = N_1(\alpha + \beta) \quad \implies \quad \alpha + \beta = \frac{1}{N_1} \sum_{i \in 1} y_i. \quad (\text{A})$$

Subtract (NE- β) from (NE- α):

$$\sum_{i=1}^N y_i - \sum_{i \in 1} y_i = N\alpha + \beta N_1 - (\alpha N_1 + \beta N_1) = (N - N_1)\alpha = N_0\alpha.$$

But $\sum_{i=1}^N y_i - \sum_{i \in 1} y_i = \sum_{i \in 0} y_i$. Hence

$$\alpha = \frac{1}{N_0} \sum_{i \in 0} y_i. \quad (\text{B})$$

Therefore, the OLS estimators are

$$\hat{\alpha} = \frac{1}{N_0} \sum_{i \in 0} y_i, \quad \hat{\alpha} + \hat{\beta} = \frac{1}{N_1} \sum_{i \in 1} y_i.$$

In other words, $\hat{\alpha}$ is the sample mean of group 0, and $\hat{\beta}$ is the difference in sample means $\bar{y}_1 - \bar{y}_0$.

(b) i. **First-order conditions for the 3-group model.** *Model:*

$$y_i = \alpha + \beta_1 D_{1i} + \beta_2 D_{2i} + u_i, \quad D_{1i} = 1\{i \in 1\}, \quad D_{2i} = 1\{i \in 2\}.$$

Groups are mutually exclusive: for every i , exactly one of $\{i \in 0, i \in 1, i \in 2\}$ holds, so

$$D_{1i}^2 = D_{1i}, \quad D_{2i}^2 = D_{2i}, \quad D_{1i}D_{2i} = 0,$$

and let $N_g = \#\{i : i \in g\}$ for $g \in \{0, 1, 2\}$ with $N = N_0 + N_1 + N_2$.

Minimize the sum of squared residuals

$$S(\alpha, \beta_1, \beta_2) = \sum_{i=1}^N (y_i - \alpha - \beta_1 D_{1i} - \beta_2 D_{2i})^2.$$

FOC w.r.t. α :

$$\frac{\partial S}{\partial \alpha} = -2 \sum_{i=1}^N (y_i - \alpha - \beta_1 D_{1i} - \beta_2 D_{2i}) = 0$$

$$\implies \sum_{i=1}^N y_i = N\alpha + \beta_1 \sum_{i=1}^N D_{1i} + \beta_2 \sum_{i=1}^N D_{2i} = N\alpha + \beta_1 N_1 + \beta_2 N_2. \quad (\text{NE-}\alpha)$$

FOC w.r.t. β_1 :

$$\frac{\partial S}{\partial \beta_1} = -2 \sum_{i=1}^N D_{1i} (y_i - \alpha - \beta_1 D_{1i} - \beta_2 D_{2i}) = 0$$

$$\implies \sum_{i=1}^N D_{1i} y_i = \alpha \sum_{i=1}^N D_{1i} + \beta_1 \sum_{i=1}^N D_{1i}^2 + \beta_2 \sum_{i=1}^N D_{1i} D_{2i}.$$

Using $D_{1i}^2 = D_{1i}$ and $D_{1i}D_{2i} = 0$,

$$\sum_{i \in 1} y_i = N_1\alpha + \beta_1 N_1 \implies \sum_{i \in 1} y_i = N_1(\alpha + \beta_1). \quad (\text{NE-}\beta_1)$$

FOC w.r.t. β_2 :

$$\begin{aligned} \frac{\partial S}{\partial \beta_2} &= -2 \sum_{i=1}^N D_{2i} (y_i - \alpha - \beta_1 D_{1i} - \beta_2 D_{2i}) = 0 \\ \implies \sum_{i=1}^N D_{2i} y_i &= \alpha \sum_{i=1}^N D_{2i} + \beta_1 \sum_{i=1}^N D_{1i} D_{2i} + \beta_2 \sum_{i=1}^N D_{2i}^2. \end{aligned}$$

Using $D_{2i}^2 = D_{2i}$ and $D_{1i}D_{2i} = 0$,

$$\sum_{i \in 2} y_i = N_2\alpha + \beta_2 N_2 \implies \sum_{i \in 2} y_i = N_2(\alpha + \beta_2). \quad (\text{NE-}\beta_2)$$

The three normal equations (NE- α), (NE- β_1), and (NE- β_2) are the FOCs that define the OLS estimator for the 3-group dummy regression.

- ii. **Solving the FOCs to get the sample estimates.** Let $N_g = \#\{i : i \in g\}$ for $g \in \{0, 1, 2\}$ and define the group means

$$\bar{y}_g = \frac{1}{N_g} \sum_{i \in g} y_i.$$

From the FOCs in part (i) we have

$$\sum_{i=1}^N y_i = N\alpha + \beta_1 N_1 + \beta_2 N_2, \quad (\text{NE-}\alpha) \quad (2.1)$$

$$\sum_{i \in 1} y_i = N_1(\alpha + \beta_1), \quad (\text{NE-}\beta_1) \quad (2.2)$$

$$\sum_{i \in 2} y_i = N_2(\alpha + \beta_2). \quad (\text{NE-}\beta_2) \quad (2.3)$$

Divide (2.2) and (2.3) by N_1 and N_2 :

$$\alpha + \beta_1 = \frac{1}{N_1} \sum_{i \in 1} y_i = \bar{y}_1, \quad \alpha + \beta_2 = \frac{1}{N_2} \sum_{i \in 2} y_i = \bar{y}_2.$$

Substitute $\beta_1 = \bar{y}_1 - \alpha$ and $\beta_2 = \bar{y}_2 - \alpha$ into (2.1):

$$\begin{aligned} \sum_{i=1}^N y_i &= N\alpha + N_1(\bar{y}_1 - \alpha) + N_2(\bar{y}_2 - \alpha) \\ &= N_0\alpha + N_1\bar{y}_1 + N_2\bar{y}_2. \end{aligned}$$

But by definition of the total sample mean decomposed by groups,

$$\sum_{i=1}^N y_i = N_0 \bar{y}_0 + N_1 \bar{y}_1 + N_2 \bar{y}_2.$$

Equate the two expressions and cancel $N_1 \bar{y}_1 + N_2 \bar{y}_2$:

$$N_0 \bar{y}_0 = N_0 \alpha \implies \hat{\alpha} = \bar{y}_0 = \frac{1}{N_0} \sum_{i \in 0} y_i.$$

Finally,

$$\hat{\alpha} + \hat{\beta}_1 = \bar{y}_1 = \frac{1}{N_1} \sum_{i \in 1} y_i, \quad \hat{\alpha} + \hat{\beta}_2 = \bar{y}_2 = \frac{1}{N_2} \sum_{i \in 2} y_i,$$

so

$$\hat{\beta}_1 = \bar{y}_1 - \bar{y}_0, \quad \hat{\beta}_2 = \bar{y}_2 - \bar{y}_0.$$

Thus $\hat{\alpha}$ is the mean of group 0, and each $\hat{\beta}_g$ is the difference between group g 's mean and the group-0 mean.

3 Residualized Regression (Frisch–Waugh)

In lecture 4, we showed that the j th row of the population regression coefficient β^* from the model

$$y_i = x_i' \beta^* + u_i$$

can be obtained by first getting the residual from an auxiliary regression of x_{ji} on all the other x 's:

$$x_{ji} = x'_{(\sim j)i} \pi + \xi_i$$

then forming:

$$\beta_j^* = \mathbb{E}[\xi_i^2]^{-1} \mathbb{E}[\xi_i y_i].$$

Suppose that we also “residualized” the dependent variable using an auxiliary regression of y_i on all the other x 's:

$$y_i = x'_{(\sim j)i} \lambda + \phi_i$$

Show that it is also true that:

$$\beta_j^* = \mathbb{E}[\xi_i^2]^{-1} \mathbb{E}[\xi_i \phi_i].$$

(In other words, we could residualize *both* x_{ji} and y_i and still get the same right answer).

Solution. Fix $j \in \{1, \dots, K\}$. Let $x_i = (x_{1i}, \dots, x_{Ki})'$ and write $x_{(\sim j)i}$ for the vector x_i with the j th coordinate removed. The population regression is

$$y_i = x_i' \beta^* + u_i, \quad \mathbb{E}[x_i u_i] = 0.$$

Consider the linear projection of x_{ji} on the other regressors:

$$x_{ji} = x_{(\sim j)i}' \pi + \xi_i,$$

where ξ_i is the residual. The FOC for π give the orthogonality conditions

$$\mathbb{E}[x_{(\sim j)i} \xi_i] = 0, \quad \mathbb{E}[\xi_i] = 0.$$

We then multiply the main model by ξ_i and take expectations as follows:

$$\mathbb{E}[\xi_i y_i] = \mathbb{E}[\xi_i (x_i' \beta^* + u_i)] = \sum_{m=1}^K \beta_m^* \mathbb{E}[\xi_i x_{mi}] + \mathbb{E}[\xi_i u_i].$$

By the auxiliary FOC, $\mathbb{E}[\xi_i x_{mi}] = 0$ for every $m \neq j$. Also,

$$\mathbb{E}[\xi_i u_i] = \mathbb{E}[(x_{ji} - x_{(\sim j)i}' \pi) u_i] = \mathbb{E}[x_{ji} u_i] - \pi' \mathbb{E}[x_{(\sim j)i} u_i] = 0 - \pi' \cdot 0 = 0,$$

because $\mathbb{E}[x_i u_i] = 0$. Hence

$$\mathbb{E}[\xi_i y_i] = \beta_j^* \mathbb{E}[\xi_i x_{ji}].$$

We then identify $\mathbb{E}[\xi_i x_{ji}]$ as $\mathbb{E}[\xi_i^2]$. Then, by using $x_{ji} = x_{(\sim j)i}' \pi + \xi_i$,

$$\mathbb{E}[\xi_i x_{ji}] = \mathbb{E}[\xi_i (x_{(\sim j)i}' \pi + \xi_i)] = \pi' \mathbb{E}[\xi_i x_{(\sim j)i}] + \mathbb{E}[\xi_i^2] = 0 + \mathbb{E}[\xi_i^2].$$

Therefore

$$\beta_j^* = \mathbb{E}[\xi_i^2]^{-1} \mathbb{E}[\xi_i y_i]$$

which is the Frisch–Waugh formula stated in the slide.

We also residualize y as well. Let $y_i = x_{(\sim j)i}' \lambda + \phi_i$ be the linear projection of y on $x_{(\sim j)}$, with residual ϕ_i . Then $\mathbb{E}[x_{(\sim j)i} \phi_i] = 0$ and

$$\mathbb{E}[\xi_i \phi_i] = \mathbb{E}[\xi_i (y_i - x_{(\sim j)i}' \lambda)] = \mathbb{E}[\xi_i y_i] - \lambda' \mathbb{E}[\xi_i x_{(\sim j)i}] = \mathbb{E}[\xi_i y_i].$$

Thus the same coefficient can be written as:

$$\beta_j^* = \mathbb{E}[\xi_i^2]^{-1} \mathbb{E}[\xi_i \phi_i]$$

In this way, we show that you can residualize both x_{ji} and y_i on the other x 's and regress one residual on the other.

4 Oaxaca Decomposition

The goal of this problem is to extend the analysis of public–private pay gaps in Lecture 3 to *male workers*. The file `pay.csv` contains 45,404 observations on male workers, age 30–50, with education of 12 or more years, in the March 2012 and 2013 CPS files. The variables are:

- `govt` = 1 if government worker
- `logwage` = log hourly wage based on earnings, weeks worked, and average hours per week last year
- `educ` = years of education
- `hs` = 1 if high school grad (12 years of schooling)
- `somecoll` = 1 if education is 13–14 years
- `college` = 1 if BA (16 years of education)
- `ma` = 1 if masters degree (18 years of schooling)
- `phd` = 1 if PhD or LLD or medical degree (20 years of schooling)
- `age` = age in years (range 30–50)

Note. `somecoll` combines two education levels (13 and 14 years). (13 years are people with some college but no degree; 14 years are people with an AA degree from a community college.)

Please create two new variables: `educ13` = $1\{\text{educ} = 13\}$ and `educ14` = $1\{\text{educ} = 14\}$ so you will have a total of 6 possible categories for education: `phd`, `ma`, `college`, `educ14`, `educ13`, `hs`.

Preliminary Setup. We begin by importing the necessary Python packages and setting the working directory.

```

1  # Packages we might need
2  import pandas as pd    # data manipulation
3  import os              # setting directory
4  import statsmodels.formula.api as smf # OLS regressions
5  import numpy as np     # arrays and matrices
6
7  print(os.getcwd())
8  # os.chdir('your_directory_here') # set working directory

```

Listing 1: Python setup: packages and working directory.

Data Preparation. We next load the dataset, create the two new education dummies `educ13` and `educ14`, and then define a categorical variable `educ_level` with 6 categories.

```

1  # Load dataset (using pandas "pd")
2  pay = pd.read_csv("pay.csv")    # adjust path if necessary
3
4  # Create 2 new variables: educ13, educ14
5  pay['educ13'] = 1 * (pay['educ'] == 13)
6  pay['educ14'] = 1 * (pay['educ'] == 14)
7  print(pay.head(2))
8
9  # Define variable for 6 education levels:
10 # 1=hs, 2=educ13, 3=educ14, 4=college, 5=ma, 6=phd
11 pay['educ_level'] = (
12     1 * (pay['hs'] == 1)
13     + 2 * (pay['educ13'] == 1)
14     + 3 * (pay['educ14'] == 1)
15     + 4 * (pay['college'] == 1)
16     + 5 * (pay['ma'] == 1)
17     + 6 * (pay['phd'] == 1)
18 )
19
20 print(pay['educ_level'].value_counts(dropna=False))

```

Output (excerpt).

	age	educ	somecoll	college	ma	govt	educ13	educ14
0	39	14	2	0	0	1	0	1
1	39	14	2	0	0	0	0	1

```

educ_level
1    11265
2     1418
3      575
4    23837
5     4311
6     1998
Name: count, dtype: int64

```

1. Construct a table like the 1st table (page 7) in Lecture 3. Show the means of all the variables for workers in the private (`govt=0`) and public (`govt=1`) sectors, and the differences in these means.

Solution.

```

1  import numpy as np
2  import pandas as pd
3
4  # Which rows (variables) to show, and their labels
5  vars_to_show = ["logwage", "educ", "hs", "educ13", "educ14", "college", "ma",
6  ↪ "phd"]
7  row_labels = {
8      "logwage": "Mean Log Wage",
9      "educ":    "Mean Education",
10     "hs":      "Fraction HS (12)",
11     "educ13":  "Fraction 13 years",
12     "educ14":  "Fraction 14 years",
13     "college": "Fraction BA (16)",
14     "ma":      "Fraction MA (18)",
15     "phd":     "Fraction PhD (20)",
16 }
17
18 # Group means for private (govt=0) and government (govt=1)
19 gm = (
20     pay.groupby("govt", observed=True)[vars_to_show]
21     .mean()
22     .T
23     .rename(index=row_labels)
24 )
25
26 # Build lecture-style wide table with grouped headers
27 cols = pd.MultiIndex.from_tuples([
28     ("Private Sector Workers", "Mean"),
29     ("Government Workers",    "Mean"),
30     ("Public{Private Gap",    ""), # gap = govt - private
31 ])
32 table1_pretty = pd.DataFrame(index=gm.index, columns=cols)
33 table1_pretty[("Private Sector Workers", "Mean")] = gm[0]
34 table1_pretty[("Government Workers", "Mean")] = gm[1]
35 table1_pretty[("Public{Private Gap", "")] = gm[1] - gm[0]
36
37 # Add sample-size row
38 n_by_sector = pay.groupby("govt", observed=True).size()
39 all_row = pd.Series({
40     ("Private Sector Workers", "Mean"): n_by_sector.get(0, np.nan),
41     ("Government Workers",    "Mean"):  n_by_sector.get(1, np.nan),
42     ("Public{Private Gap",    ""):      np.nan,
43 }, name="Sample Size")
44
45 # Final touches: order, labels, rounding
46 table1_pretty.index.name = " "
47 table1_pretty = pd.concat([table1_pretty, all_row.to_frame().T], axis=0)
48 num_rows = table1_pretty.index != "Sample Size"
49 table1_pretty.loc[num_rows] = table1_pretty.loc[num_rows].astype(float).round(3)
50 table1_pretty.loc["Sample Size"] = table1_pretty.loc["Sample
51 ↪ Size"].astype("Int64")

```

11

Table 1: Means of variables by sector (govt= 1 vs govt= 0) and public-private gap (govt-private). Entries rounded to three decimals; sample sizes shown as counts.

	Private Mean	Govt Mean	Gap (Govt-Priv)
Mean Log Wage	3.087	3.299	0.212
Mean Education	13.287	13.618	0.331
Fraction HS (12)	0.337	0.173	-0.164
Fraction 13 years	0.089	0.191	0.102
Fraction 14 years	0.083	0.081	-0.002
Fraction BA (16)	0.264	0.305	0.041
Fraction MA (18)	0.173	0.181	0.008
Fraction PhD (20)	0.059	0.086	0.027
Sample Size	38 553.000	6 681.000	

- Construct a table like the 2nd table (page 8) in Lecture 3. Show the fractions of private and government workers at each education level, and the mean wages of the two groups in each education level, and the public-private wage gap at each level.

```

1  # --- Table 2: Education-specific composition and wage gaps ---
2  import numpy as np
3  import pandas as pd
4  from IPython.display import display
5
6  # (Optional) keep wide tables on one line in VS Code / Jupyter
7  pd.set_option('display.expand_frame_repr', False)
8
9  # 1) Define a single education code: 12, 13, 14, 16, 18, 20
10 edu_code = np.select(
11     [
12         pay['hs'].eq(1), pay['educ13'].eq(1), pay['educ14'].eq(1),
13         pay['college'].eq(1), pay['ma'].eq(1), pay['phd'].eq(1),
14     ],
15     [12, 13, 14, 16, 18, 20],
16     default=np.nan
17 )
18 cats = [12, 13, 14, 16, 18, 20]
19 pay['edu_code'] = pd.Categorical(edu_code, categories=cats, ordered=True)
20
21 # 2) Drop invalid rows
22 df = pay.dropna(subset=['edu_code']).copy()
23
24 # 3) Group by (education, sector): counts + mean log wages
25 g = df.groupby(['edu_code', 'govt'], observed=True)
26 by_cell = g.agg(n=('logwage', 'size'), mean_logw=('logwage', 'mean')).reset_index()

```

Listing 3: Code (Part 1): Setup, education codes, grouping.

```

1  # 4) Fractions within each sector
2  by_cell['sector_total'] = by_cell.groupby('govt')['n'].transform('sum')
3  by_cell['fraction_in_group'] = by_cell['n'] / by_cell['sector_total']
4
5  # 5) Pivot to wide
6  wide_w = by_cell.pivot(index='edu_code', columns='govt', values='mean_logw')
7  wide_f = by_cell.pivot(index='edu_code', columns='govt',
8      ↪ values='fraction_in_group')
9
10 # 6) Assemble table
11 cols = pd.MultiIndex.from_tuples([
12     ('Private Sector Workers', 'Mean Log Wage'),
13     ('Private Sector Workers', 'Fraction in Group'),
14     ('Government Workers', 'Mean Log Wage'),
15     ('Government Workers', 'Fraction in Group'),
16     ('Govt Wage Premium', '')
17 ])
18 table2 = pd.DataFrame({
19     ('Private Sector Workers', 'Mean Log Wage'): wide_w.get(0),
20     ('Private Sector Workers', 'Fraction in Group'): wide_f.get(0),
21     ('Government Workers', 'Mean Log Wage'): wide_w.get(1),
22     ('Government Workers', 'Fraction in Group'): wide_f.get(1),
23 })
24 table2[('Govt Wage Premium', '')] = table2[('Government Workers', 'Mean Log Wage')]
25 ↪ - table2[('Private Sector Workers', 'Mean Log Wage')]
26
27 # 7) Add "All" row
28 overall = df.groupby('govt').agg(n=('logwage', 'size'),
29     ↪ mean_logw=('logwage', 'mean'))
30 all_row = pd.Series({
31     ('Private Sector Workers', 'Mean Log Wage'): overall.loc[0, 'mean_logw'],
32     ('Private Sector Workers', 'Fraction in Group'): 1.0,
33     ('Government Workers', 'Mean Log Wage'): overall.loc[1, 'mean_logw'],
34     ('Government Workers', 'Fraction in Group'): 1.0,
35     ('Govt Wage Premium', ''): overall.loc[1, 'mean_logw'] -
36     ↪ overall.loc[0, 'mean_logw']
37 }, name='All')
38 table2 = pd.concat([table2, all_row.to_frame().T], axis=0)
39 table2.index.name = 'Education'
40 table2 = table2.round(3)
41
42 display(table2)

```

Listing 4: Code (Part 2): Fractions, wide pivot, and final wage gap table.

Table 2: Education-specific composition and wage gaps by sector. Entries rounded to three decimals. “Govt Wage Premium” is government minus private.

Education	Private Sector Workers		Government Workers		Govt Wage Prem.
	Mean Log Wage	Frac. in Group	Mean Log Wage	Frac. in Group	
12	2.703	0.307	2.968	0.192	0.265
13	2.873	0.113	3.123	0.250	0.250
14	2.935	0.048	3.157	0.121	0.222
16	3.354	0.246	3.483	0.258	0.129
18	3.542	0.043	3.413	0.069	−0.129
20	3.692	0.040	3.543	0.067	−0.149
All	3.007	1.000	3.219	1.000	0.212

3. Using the notation of Lecture 3, let private sector workers be called “group a ” (the reference group), let government workers be called “group b ”, and define the 6 category variables D_{gi} for education level $g = 1, \dots, 6$. Let

$$x_i^a = (D_{1i}, D_{2i}, \dots, D_{6i})'.$$

Find $\bar{x}^a$ and $\bar{x}^b$.

Setup for Part 3. Let private-sector workers be “group a ” ($govt = 0$) and government workers be “group b ” ($govt = 1$). Define six education dummies D_{gi} for $g \in \{12, 13, 14, 16, 18, 20\}$. For each group, compute the within-group means of these dummies, $\bar{x}^a$ and $\bar{x}^b$, which are the shares of each education level in groups a and b .

```

1  # Groups and education dummies; compute  $\bar{x}^a$  and  $\bar{x}^b$  once
2  pay["group"] = np.where(pay["govt"].eq(0), "a", "b")
3
4  # six education indicators
5  pay["D_12"] = (pay["hs"]==1).astype(int)
6  pay["D_13"] = (pay["educ"]==13).astype(int)
7  pay["D_14"] = (pay["educ"]==14).astype(int)
8  pay["D_16"] = (pay["college"]==1).astype(int)
9  pay["D_18"] = (pay["ma"]==1).astype(int)
10 pay["D_20"] = (pay["phd"]==1).astype(int)
11 D_cols = ["D_12", "D_13", "D_14", "D_16", "D_18", "D_20"]
12
13 # within{group means (shares)
14 xbar_a = pay.loc[pay["group"].eq("a"), D_cols].mean().rename("Private share
15 ↪ ( $\bar{x}^a$ )")
16 xbar_b = pay.loc[pay["group"].eq("b"), D_cols].mean().rename("Govt share ( $\bar{x}^b$ )")
17 xbar_tbl = (pd.concat([xbar_a, xbar_b], axis=1)
18               .rename(index={"D_12": "12", "D_13": "13", "D_14": "14",
19                               "D_16": "16", "D_18": "18", "D_20": "20"})
20               .round(3))
21 display(xbar_tbl)

```

Listing 5: Code: groups, education dummies, and within-group shares $\bar{x}^a, \bar{x}^b$.

Table 3: Within-group education shares. $\bar{x}^a$ and $\bar{x}^b$ are the means of D_{gi} inside each group (equal to the “Fraction in Group” columns from Table 2 for the level rows).

Education	Private share ($\bar{x}^a$)	Govt share ($\bar{x}^b$)
12	0.337	0.192
13	0.180	0.182
14	0.113	0.121
16	0.246	0.258
18	0.083	0.180
20	0.040	0.067

- (a) Fit an OLS regression model for wages on x_i for each group, and estimate the coefficients $\hat{\beta}^a$ and $\hat{\beta}^b$. Verify that these coefficients are the mean log wages of groups a and b , respectively, for each level of education.

```

1  # --- Part 3(a): OLS of log wages on education dummies, by group ---
2
3  import numpy as np
4  import pandas as pd
5  import statsmodels.formula.api as smf
6
7  # Ensure educ13/educ14 exist and create categorical edu_code
8  if 'educ13' not in pay.columns:
9      pay['educ13'] = (pay['educ'] == 13).astype(int)
10 if 'educ14' not in pay.columns:
11     pay['educ14'] = (pay['educ'] == 14).astype(int)
12
13 if 'edu_code' not in pay.columns:
14     edu_code = np.select(
15         [
16             pay['hs'].eq(1),
17             pay['educ13'].eq(1),
18             pay['educ14'].eq(1),
19             pay['college'].eq(1),
20             pay['ma'].eq(1),
21             pay['phd'].eq(1),
22         ],
23         [12, 13, 14, 16, 18, 20],
24         default=np.nan
25     )
26     pay['edu_code'] = pd.Categorical(
27         edu_code, categories=[12,13,14,16,18,20], ordered=True
28     )
29
30 # Split into private (a) and government (b)
31 a = pay.loc[pay['govt'] == 0].copy()
32 b = pay.loc[pay['govt'] == 1].copy()
33
34 # OLS with no intercept: coefficients = cell means
35 formula = "logwage ~ 0 + D_12 + D_13 + D_14 + D_16 + D_18 + D_20"
36 fit_a = smf.ols(formula, data=a).fit(cov_type='HC1')
37 fit_b = smf.ols(formula, data=b).fit(cov_type='HC1')
38
39 # Extract coefficients and match to edu codes
40 beta_a = fit_a.params.rename(index=lambda s: int(s.split('_')[1])).sort_index()
41 beta_b = fit_b.params.rename(index=lambda s: int(s.split('_')[1])).sort_index()
42
43 # Direct means for verification
44 mean_a = a.groupby('edu_code',
45     ↪ observed=True)['logwage'].mean().reindex(beta_a.index)
46 mean_b = b.groupby('edu_code',
47     ↪ observed=True)['logwage'].mean().reindex(beta_b.index)

```

```

1  # Sanity check: coefficients group means
2  assert np.allclose(beta_a.values, mean_a.values)
3  assert np.allclose(beta_b.values, mean_b.values)
4
5  # Display tidy comparison table
6  out_a_b = pd.DataFrame({
7      'OLS coef (a)': beta_a.round(3),
8      'Mean log wage (a)': mean_a.round(3),
9      'OLS coef (b)': beta_b.round(3),
10     'Mean log wage (b)': mean_b.round(3),
11 })
12 out_a_b.index.name = 'Education'
13 display(out_a_b)

```

Listing 6: Code: OLS of log wages on education dummies by group.

Table 4: OLS coefficients and direct mean log wages, by education level and sector. Coefficients equal the cell means, as expected.

Education	OLS coef (a)	Mean log wage (a)	OLS coef (b)	Mean log wage (b)
12	2.703	2.703	2.968	2.968
13	2.873	2.873	3.123	3.123
14	2.935	2.935	3.157	3.157
16	3.354	3.354	3.483	3.483
18	3.542	3.542	3.413	3.413
20	3.692	3.692	3.543	3.543

(b) Using your estimates of $\bar{x}^a$, $\bar{x}^b$, $\hat{\beta}^a$ and $\hat{\beta}^b$, construct

$$\bar{y}^a = \sum_g p_g^a y_g^a = (\bar{x}^a)' \hat{\beta}^a, \quad \bar{y}^b = \sum_g p_g^b y_g^b = (\bar{x}^b)' \hat{\beta}^b,$$

and verify that these match the means of wages for groups a and b that you construct directly.

```

1  # --- Q3(b) Part 1: setup, dummies, and group OLS ---
2
3  import numpy as np
4  import pandas as pd
5  import statsmodels.formula.api as smf
6
7  # 1) Split groups: a = private (govt=0), b = government (govt=1)
8  a = pay.loc[pay['govt'].eq(0)].copy()
9  b = pay.loc[pay['govt'].eq(1)].copy()
10
11 # 2) Education-category dummies (D_12,...,D_20).
12 #    (If created earlier, this just reasserts them.)
13 pay['D_12'] = (pay['hs'] == 1).astype(int)
14 pay['D_13'] = (pay['educ'] == 13).astype(int)
15 pay['D_14'] = (pay['educ'] == 14).astype(int)
16 pay['D_16'] = (pay['college'] == 1).astype(int)
17 pay['D_18'] = (pay['ma'] == 1).astype(int)
18 pay['D_20'] = (pay['phd'] == 1).astype(int)
19
20 # align dummies to a/b rows
21 D_cols = ['D_12', 'D_13', 'D_14', 'D_16', 'D_18', 'D_20']
22 a[D_cols] = pay.loc[a.index, D_cols]
23 b[D_cols] = pay.loc[b.index, D_cols]
24
25 # 3) Dummy-only OLS (no intercept each coefficient = cell mean)
26 mod_a = smf.ols("logwage ~ 0 + D_12 + D_13 + D_14 + D_16 + D_18 + D_20",
27 ↪ data=a).fit(cov_type='HC1')
28 mod_b = smf.ols("logwage ~ 0 + D_12 + D_13 + D_14 + D_16 + D_18 + D_20",
29 ↪ data=b).fit(cov_type='HC1')
30
31 beta_a = mod_a.params[D_cols]      # \hat{\alpha}_g = mean log wage in group a, cell g
32 beta_b = mod_b.params[D_cols]      # \hat{\beta}_g = mean log wage in group b, cell g

```

Listing 7: Code (Part 1): Setup, dummies, and group-specific OLS (cell means).

```

1  # --- Q3(b) Part 2: shares, weighted averages, and checks ---
2
3  # 4) Within-group education shares  \bar{x}^a and \bar{x}^b
4  xbar_a = a[D_cols].mean()    # p_g^a
5  xbar_b = b[D_cols].mean()    # p_g^b
6
7  # 5) Weighted-average means:  \bar{y}^a = (\bar{x}^a)' \hat{\beta}^a,  \bar{y}^b =
   ↪  (\bar{x}^b)' \hat{\beta}^b
8  ybar_a_weighted = float(np.dot(xbar_a.values, beta_a.values))
9  ybar_b_weighted = float(np.dot(xbar_b.values, beta_b.values))
10
11 # 6) Direct sample means for verification
12 ybar_a_direct = a['logwage'].mean()
13 ybar_b_direct = b['logwage'].mean()
14
15 # 7) Neat comparison table
16 check = pd.DataFrame({
17     "Weighted avg (x)": [ybar_a_weighted, ybar_b_weighted],
18     "Direct mean logwage": [ybar_a_direct, ybar_b_direct],
19     "Difference": [ybar_a_weighted - ybar_a_direct,
20                  ybar_b_weighted - ybar_b_direct]
21 }, index=pd.Index(['Group a (private)', 'Group b (government)'],
   ↪  name='Group')).round(3)
22
23 display(check)
24
25 # Optional: assert equality up to rounding
26 assert np.isclose(ybar_a_weighted, ybar_a_direct)
27 assert np.isclose(ybar_b_weighted, ybar_b_direct)

```

Listing 8: Code (Part 2): Shares, weighted mean log wages, and equality check to direct means.

Table 5: Weighted-average mean log wages equal the direct sample means (law of iterated expectations).

	Weighted avg ($\bar{x}'\hat{\beta}$)	Direct mean logwage	Difference
Group a (private)	3.007	3.007	0.000
Group b (government)	3.219	3.219	0.000

(c) Using your estimates of $\bar{x}^a$ and $\hat{\beta}^b$ construct

$$\bar{y}_{\text{counterf}}^b = \sum_g p_g^a y_g^b = (\bar{x}^a)' \hat{\beta}^b.$$

Find the adjusted wage gap: $\bar{y}_{\text{counterf}}^b - \bar{y}^a$. How does this compare to the actual gap $\bar{y}^b - \bar{y}^a$?

```

1  # --- 3(c) Counterfactual using a's composition and b's returns ---
2
3  import numpy as np
4  import pandas as pd
5  import statsmodels.formula.api as smf
6
7  # Dummies (should already exist from part 3 setup)
8  D_cols = ['D_12', 'D_13', 'D_14', 'D_16', 'D_18', 'D_20']
9  order = [f'D_{g}' for g in [12,13,14,16,18,20]]
10
11 # Split data
12 a = pay.loc[pay['govt'].eq(0)].copy() # private (group a)
13 b = pay.loc[pay['govt'].eq(1)].copy() # government (group b)
14
15 # 1) Returns for group b:  $\hat{\beta}_b$  from a no-constant dummy regression
16 formula_b = "logwage ~ 0 + " + " + ".join(D_cols)
17 fit_b = smf.ols(formula_b, data=b).fit(cov_type='HC1')
18 beta_b = fit_b.params.reindex(order)
19
20 # 2) Composition for group a:  $\bar{x}_a$  (mean of education dummies)
21 xbar_a = a[D_cols].mean().reindex(order)
22
23 # 3) Counterfactual mean for group b under a's composition
24 ybar_b_counterf = float(np.dot(xbar_a.values, beta_b.values))
25
26 # 4) Actual means and gaps
27 ybar_a = a['logwage'].mean()
28 ybar_b = b['logwage'].mean()
29
30 gap_actual = ybar_b - ybar_a
31 gap_adjusted = ybar_b_counterf - ybar_a

```

Listing 9: Code (Part 1): Counterfactual setup, returns, composition, and adjusted gap.

```

1  # (Optional) Oaxaca-style split when using b-coefficients as reference
2  # composition piece:  $(\bar{x}^b - \bar{x}^a)' * \beta^b$ 
3  xbar_b = b[D_cols].mean().reindex(order)
4  composition = float(np.dot((xbar_b - xbar_a).values, beta_b.values))
5
6  # returns piece:  $(\beta^b - \beta^a)' * \bar{x}^a$ 
7  fit_a = smf.ols("logwage ~ 0 + " + " + ".join(D_cols),
8  ↪ data=a).fit(cov_type='HC1')
9  beta_a = fit_a.params.reindex(order)
10 returns = float(np.dot((beta_b - beta_a).values, xbar_a.values))
11
12 # 5) Display tidy summary
13 out = pd.DataFrame(
14     {
15         "Mean log wage": [ybar_a, ybar_b, ybar_b_counterf],
16         "|": ["", "", ""],
17         "Gap vs group a": [0.0, gap_actual, gap_adjusted],
18     },
19     index=["Group a (private)", "Group b (govt) { actual", "Group b {
20 ↪ counterfactual"]
21 )
22
23 pieces = pd.DataFrame(
24     {
25         "Composition piece (using  $\hat{\beta}$ )": [composition],
26         "Returns piece (using  $\hat{x}$ )": [returns],
27         "Composition + Returns": [composition + returns],
28         "Actual gap  $y^b - y^a$ ": [gap_actual],
29     }
30 )
31 display(out.round(3))
display(pieces.round(3))

```

Listing 10: Code (Part 2): Oaxaca-style decomposition and summary output.

Table 6: Counterfactual with private-sector composition and government returns (Part 3(c)).

Panel A. Mean log wages and gaps		
Group	Mean log wage	Gap vs group a
Group a (private)	3.007	0.000
Group b (government) — actual	3.219	0.212
Group b — counterfactual	3.154	0.147
Panel B. Oaxaca-style decomposition (using $\hat{\beta}^b$ as reference)		
Component	Value	
Composition effect $(\bar{x}^b - \bar{x}^a)' \hat{\beta}^b$	0.065	
Returns effect $(\hat{\beta}^b - \hat{\beta}^a)' \bar{x}^a$	0.147	
Composition + Returns	0.212	
Actual gap $\bar{y}^b - \bar{y}^a$	0.212	

Notes: Panel A reports observed and counterfactual mean log wages when group b (government) is evaluated at the private-sector education mix $\bar{x}^a$ with government returns $\hat{\beta}^b$. Panel B decomposes the total gap into a composition (education mix) piece and a returns (pay for the same education) piece; values are rounded to three decimals.

Solution.

Interpretation of Part (c). The observed public–private wage gap is

$$\bar{y}^b - \bar{y}^a = 3.219 - 3.007 = 0.212.$$

Using the Oaxaca decomposition, we compute the counterfactual mean log wage for government workers under the private-sector education distribution:

$$\bar{y}_{\text{counterf}}^b = 3.154.$$

This implies an adjusted wage gap of

$$\bar{y}_{\text{counterf}}^b - \bar{y}^a = 3.154 - 3.007 = 0.147,$$

which is smaller than the actual observed gap.

Decomposing the total gap of 0.212, we find:

- *Composition effect (education mix):* 0.065,
- *Returns effect (differences in pay for the same education):* 0.147.

Thus, approximately 31% of the wage gap can be attributed to differences in the education distribution across sectors, while the remaining 69% reflects differences in returns to education between government and private-sector employment.

(d) Construct the weight

$$w_g = \frac{N_g^a}{N_g^b}$$

for education categories $g = 1, \dots, 6$. Use this to construct

$$\bar{y}_{\text{counterf2}}^b = \frac{\sum_{i \in b} w_{g_i} y_i}{\sum_{i \in b} w_{g_i}} \quad \text{and verify that} \quad \bar{y}_{\text{counterf}}^b = \bar{y}_{\text{counterf2}}^b,$$

that is,

$$\sum_g p_g^a y_g^b = \frac{\sum_{i \in b} w_{g_i} y_i}{\sum_{i \in b} w_{g_i}}.$$

```

1  import numpy as np
2  import pandas as pd
3
4  # --- (d) Counterfactual 2: cell weights and weighted mean ---
5
6  # Counts by education cell in each group
7  Na_g = a[D_cols].sum(axis=0)    # number of a (private) in each edu cell
8  Nb_g = b[D_cols].sum(axis=0)    # number of b (govt) in each edu cell
9
10 # Weights w_g = Na_g / Nb_g (defined cell-by-cell)
11 w_g = (Na_g / Nb_g).rename('w_g')
12
13 # Attach weights to each person in group b according to their edu cell
14 # For each person i in b, w_i = sum_g w_g * D_gi (since exactly one D_gi=1)
15 w_i = pd.Series(0.0, index=b.index)
16 for col in D_cols:
17     w_i = w_i + w_g[col] * b[col]
18
19 # Weighted mean of log wages for group b under weights w_g:
20 numer = (w_i * b['logwage']).sum()
21 denom = w_i.sum()
22 ybar_b_counterf2 = numer / denom
23
24 # Compare to the counterfactual from (c): (xbar^a)' beta^b
25 # (Assumes you already computed ybar_b_counterf)
26 diff_cf = ybar_b_counterf2 - ybar_b_counterf
27
28 # Adjusted gap using the cell-weighted counterfactual
29 gap_adj_d = ybar_b_counterf2 - ybar_a_direct
30
31 # Tidy display
32 check_d = pd.DataFrame({
33     "ybar_b (counterf2)": [ybar_b_counterf2],
34     "ybar_b (xbar^a beta^b)": [ybar_b_counterf],
35     "Difference": [diff_cf],
36     "Adjusted gap (counterf2 - ybar_a)": [gap_adj_d]
37 })
38 display(check_d.round(3))

```

Listing 11: Code (Part 3d): Counterfactual 2 using cell weights and weighted means.

Table 7: Counterfactual 2 with cell weights and weighted means (Part 3(d)).

	$\bar{y}_{\text{counterf2}}^b$	$\bar{y}^b = (\bar{x}^a)' \hat{\beta}^b$	Difference	Adjusted gap $(\bar{y}_{\text{counterf2}}^b - \bar{y}^a)$
Value	3.154	3.154	0.000	0.147

Notes: Table compares the counterfactual mean log wage for government workers using cell weights $\{w_g\}$ (Column 1) to the alternative construction $(\bar{x}^a)' \hat{\beta}^b$ from Part (c) (Column 2). The near-zero difference confirms both methods are consistent. The last column reports the adjusted wage gap relative to private workers.

Solution.

Interpretation of Part (d). In Part (d), we constructed the counterfactual mean log wage for government workers using cell weights defined as

$$w_g = \frac{N_g^a}{N_g^b}, \quad g = 1, \dots, 6$$

where N_g^a and N_g^b are the numbers of workers in education category g for the private and government sectors, respectively.

The weighted counterfactual mean is

$$\bar{y}_{\text{counterf2}}^b = \frac{\sum_{i \in b} w_g y_{gi}}{\sum_{i \in b} w_g} = 3.154.$$

We then verified that this equals the alternative construction from Part (c),

$$\bar{y}_{\text{counterf2}}^b = (\bar{x}^a)' \hat{\beta}^b = 3.154,$$

with a difference of essentially 0.0, confirming that both methods are consistent.

Finally, the adjusted wage gap using this counterfactual is

$$\bar{y}_{\text{counterf2}}^b - \bar{y}^a = 3.154 - 3.007 = 0.147,$$

which matches the *returns* component identified earlier in the Oaxaca decomposition. Thus, Part (d) confirms that the decomposition result is robust to the alternative construction of the counterfactual mean.